

Stored XSS with Multiple WAF Bypasses on an SRC

By h0af3ng · Translated from the Chinese original

0x01: Stored XSS via the Editor's Hyperlink Component

I'm honestly too much of a noob for high-quality bugs, so let me casually slap together an article.

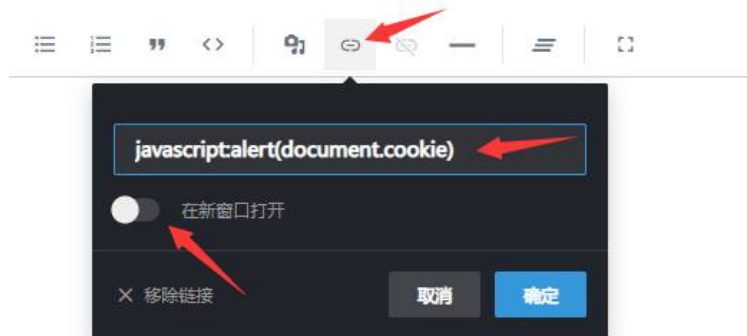
On a dark and windy night, a hacker friend woke me up to go bug hunting. I peeked at the platform and it happened to be running an event, so I thought: time to test the blade.

First, some reconnaissance. One odd-looking domain caught my attention. Visiting it revealed a submission platform where users could publish articles to a review backend.

After logging in, an "article management" section appeared right away.

My first thought was to publish an article. While exploring the system I found an editor I couldn't identify (if any expert recognizes it, please tell me) that had a hyperlink feature — so let's try to abuse it.

Inject a pseudo-protocol into the hyperlink to construct XSS:



A small detail here — the little toggle button below:

1. When ON: clicking the hyperlink opens the jump in a new window. 2. When OFF: clicking it executes JavaScript within the current site.

So we need it OFF.

After publishing the article, the JavaScript fired right in the body:



Because I inserted it as a bare hyperlink, the page itself stayed pure white.

0x02: Stored XSS in the Article Body, with Bypass

Into the new-article page, payload first. I'm 菜 enough that my XSS method is "see a box, X it".



Enter the payload in the title and body, hit submit, with Burp Suite capturing.

创建文章

标题

封面

内容

16 行高 字间距
A B I U
A⁵ A₅

常规

testtestestx">

提交

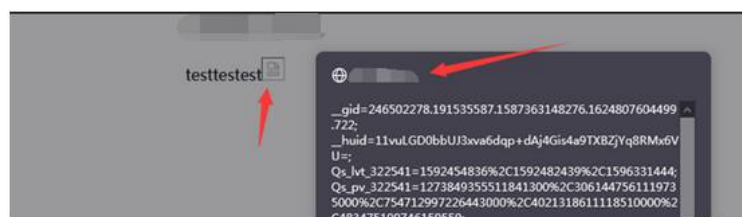
You can see the pipeline HTML-entity-encodes the payload on the front end first.

```
M8C4ATu8RIKZUdcMJnmCqnZg91lZikD%22%3B%7D
```

```
{ "title": "testtestestx\"><img src=1 onerror=alert(document.cookie)>", "content": "<p>testtestestx"&quot;&gt;&lt;img src=1 onerror=alert(document.cookie)&gt;&lt;/p>" }
```

Simply swap the payload back in at this point and the encoding is bypassed.

Visiting the published article page, the XSS fired successfully.



With two stored XSS submitted, I clocked out and went to sleep.



0x03: Stored XSS via the Editor's Media Component

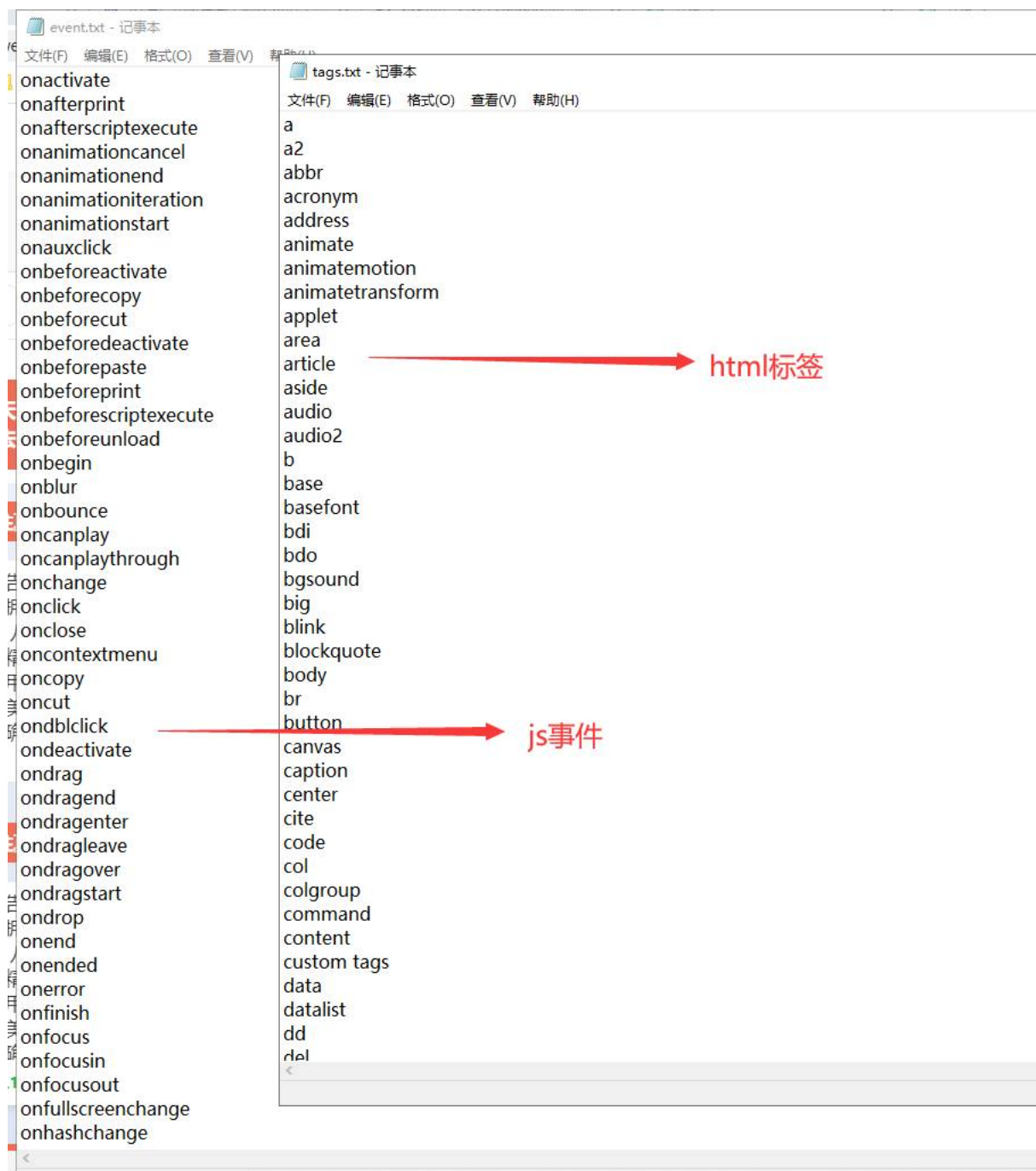
After those two stored XSS, I felt the story wasn't over. I came back the next day — and sure enough, persistence pays.

While testing the editor's other features, I found that a resource URL inserted via the media feature gets echoed onto the page.

Add a network resource:



Many tags and events were filtered, but that didn't stop us from FUZZing via Burp.

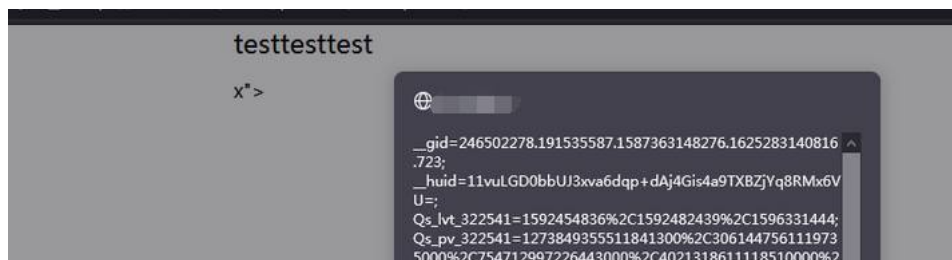


Choosing "embedded media", after repeated testing the payload looked like this:

```
x"><marquee loop=1 width=0 onfinish=alert(document.cookie)>
```

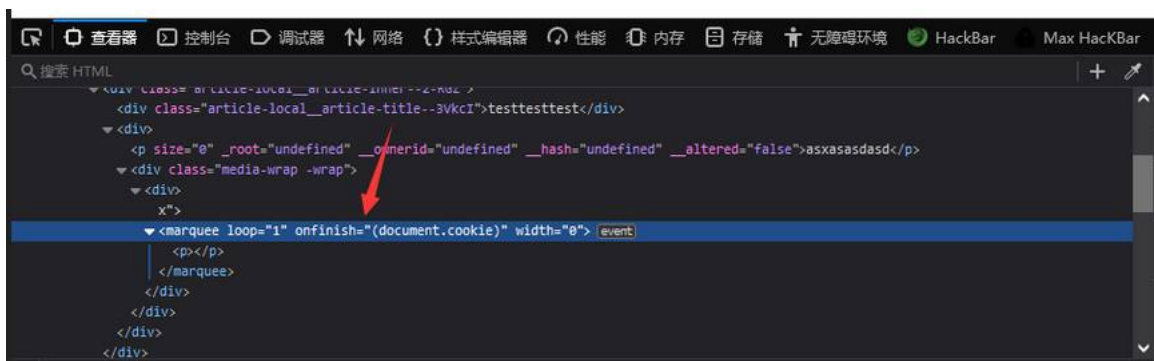


Submit the article, visit the URL — fired successfully.



0x04: Stored XSS via the Media Component (the Bypass)

The earlier bugs were all reported and fixed within days. I thought that was the end. It was not. Fixed? Then a real man attempts the bypass. Replaying the steps from 0x03, I mapped out the filtering:



1. The alert script function is filtered. 2. Quite a few JS events are filtered, but the onfinish event is not — and the tag isn't filtered either.

This developer was clearly lazy; the filtering is sloppy as hell. So we prescribe the right medicine:

```
x"><marquee loop=1 width=0 onfinish=prompt(document.cookie)>
```

And it executed, as wished.

Due to the fix status on the business side, I can't publish this site's full bypass for now.

To find out what happens next, tune in to the next episode.

0x05: A Few XSS Tips

1. When a parameter's output is wrapped in backticks inside JS, `${ }` can execute JavaScript:

```
<script>var a=`Hello${alert(1)}`</script>
```

2. XSS testing inside SVG:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<svg onload="window.location='https://www.baidu.com'">
</svg>
```

3. Against WAFs that forbid function execution, try this payload:

```
<svg onload="[['\146\151\154\164\145\162']
['\143\157\156\163\164\162\165\143\164\157\162']
['\141\154\145\162\164']()]">
```

4. During testing, try encoding bypasses: multiple layers of URL encoding, HTML-entity encoding, character splicing.

Or use carriage returns / line feeds to bypass certain WAFs — `\r\n` can do the trick:

```
<svg onload\r\n=$.globalEval("al"+"ert()");>
```